BIANCA ESTEFANY ALVES PEREIRA
NATHALIA BEATRIZ PODKOWA

TRABALHO DA AEP

MARINGÁ – PR
2026

BIANCA ESTEFANY ALVES PEREIRA
NATHALIA BEATRIZ PODKOWA

TRABALHO DA AEP

Trabalho apresentado como requisito parcial de avaliação das disciplinas cursadas no período, referente à Atividade de Estudos Práticos (AEP).

UNIVERSIDADE CESUMAR
CURSO DE ENGENHARIA DE SOFTWARE

MARINGÁ – PR
2026

SUMÁRIO

1 PRODUÇÃO TEXTUAL – METEP....4
1.1 Contextualização....4
1.2 Questão norteadora....5
1.3 Objetivo geral....5
1.4 Objetivos específicos....5
1.5 Justificativa....6
2 ALGORITMOS E LÓGICA DE PROGRAMAÇÃO....7
2.1 Fluxograma Geral....7
2.2 Pseudocódigo Geral....7
2.3 Fluxograma Específico....8
2.4 Pseudocódigo Específico....9
3 ENGENHARIA DE SOFTWARE....10
3.1 Identificação dos usuários envolvidos....10
3.2 Delimitação do Escopo do Sistema....10
3.3 Requisitos Funcionais....10
3.4 Requisitos Não Funcionais....11
3.5 Processo de desenvolvimento....11
3.6 Planejamento das Sprints....12
4 IMPLEMENTAÇÃO EM LINGUAGEM C....13
5 ENGENHARIA DE SOFTWARE – 2ª ETAPA....13
5.1 Requisitos funcionais (atualizados)....13
5.2 Requisitos não funcionais....13
5.3 Análise de riscos do projeto....14
5.4 Casos de uso....15
5.4.1 Diagrama de casos de uso....15
5.4.2 Descrição dos casos de uso principais....15
UC01 – Cadastrar ação....15
UC02 – Listar ações....16
UC03 – Pesquisar ação....16
UC04 – Atualizar situação e registrar participantes....16
UC05 – Gerar resumo....17
UC06 – Encerrar programa....17
5.5 Registro de testes....17
6 MANUAL DO USUÁRIO....18
6.1 Objetivo....18
6.2 Compilação e execução....19
6.3 Opções do menu....19
6.4 Exemplos de uso e capturas de tela....19
6.5 Mensagens de erro mais comuns....23
7 CONSIDERAÇÕES FINAIS....24
8 APRESENTAÇÃO EM VÍDEO....24
REFERÊNCIAS....24

1 PRODUÇÃO TEXTUAL – METEP

1.1 Contextualização

O Programa Saúde na Escola (PSE) é uma política pública brasileira que integra as áreas da saúde e da educação com o objetivo de contribuir para a formação integral dos estudantes da educação básica. O programa foi instituído em 2007, por meio do Decreto nº 6.286, de 5 de dezembro de 2007, sendo desenvolvido em uma ação intersetorial entre o Ministério da Saúde e o Ministério da Educação. Sua criação parte do entendimento de que a escola constitui um espaço importante para a promoção da saúde, prevenção de doenças e desenvolvimento de hábitos que contribuam para a qualidade de vida dos estudantes (BRASIL, 2007).

A criação do PSE está relacionada a um contexto histórico no qual questões de saúde passaram a exigir maior atenção no ambiente escolar. Durante a década de 1990, destacavam-se problemas como a epidemia de HIV/AIDS e o aumento do consumo de crack e outras drogas. Nesse período, também foram desenvolvidas iniciativas de aproximação entre saúde e educação, que contribuíram para a construção de políticas voltadas à prevenção e à promoção da saúde entre crianças e adolescentes. Essas experiências anteriores ajudaram a estabelecer as bases para a criação do Programa Saúde na Escola (BRASIL, 2006).

O PSE busca, portanto, superar uma visão de saúde limitada ao atendimento de doenças, promovendo ações preventivas e educativas dentro do ambiente escolar. Para isso, o programa prevê a articulação entre as Equipes de Saúde da Família e as escolas, possibilitando que profissionais das duas áreas atuem de forma integrada. Essa intersetorialidade é importante porque as condições de saúde dos estudantes estão relacionadas a diferentes fatores sociais e educacionais, tornando necessária uma atuação conjunta para enfrentar vulnerabilidades e ampliar o acesso aos serviços de saúde (CHIARI et al., 2018). Entre as ações desenvolvidas pelo programa estão atividades relacionadas ao combate ao mosquito Aedes aegypti, incentivo à prática de atividades físicas, prevenção ao uso de álcool, tabaco e outras drogas, promoção da cultura de paz e prevenção das violências, cuidados com a saúde bucal, atualização da vacinação, promoção da alimentação saudável, saúde auditiva, saúde sexual e reprodutiva e saúde ocular. Durante a pandemia de Covid-19, também foi incorporada ao programa uma ação específica de prevenção da doença nas escolas. Dessa forma, o PSE apresenta uma abordagem ampla, envolvendo diferentes aspectos que podem interferir no bem-estar e no desenvolvimento dos estudantes (BRASIL, 2017; BRASIL, 2025b).

A importância do PSE também está relacionada à possibilidade de melhorar a qualidade de vida dos estudantes e contribuir para a redução de problemas de saúde

que podem interferir em sua vida escolar. A literatura destaca como possíveis impactos a redução de agravos relacionados a infecções, doenças crônicas não transmissíveis e problemas de saúde mental, além do fortalecimento da integração entre os setores de saúde e educação. Nesse sentido, a atuação conjunta desses setores pode favorecer a construção de ambientes escolares mais saudáveis e contribuir para o desenvolvimento integral dos estudantes (SOUSA; ESPERIDIÃO; MEDINA, 2017).

Por fim, a implementação do Programa Saúde na Escola depende da participação e da articulação de diferentes atores, incluindo escolas, Equipes de Saúde da Família, municípios, Ministério da Saúde e Ministério da Educação. A participação das famílias e da sociedade também é considerada fundamental para que as ações sejam reconhecidas e tenham continuidade. Dessa maneira, o PSE representa uma estratégia de política pública que busca aproximar saúde e educação, utilizando o ambiente escolar como espaço de prevenção, promoção da saúde e desenvolvimento integral dos estudantes (VIEIRA; BELISÁRIO, 2018).

1.2 Questão norteadora

Como uma aplicação desenvolvida em linguagem C pode auxiliar no planejamento, registro e acompanhamento de ações coletivas de promoção da saúde mental no ambiente escolar, fornecendo informações organizadas para apoiar a equipe responsável e preservando a privacidade dos estudantes?

1.3 Objetivo geral

Desenvolver uma aplicação em linguagem C para auxiliar no planejamento, registro e acompanhamento de ações coletivas de promoção da saúde mental no ambiente escolar, permitindo organizar informações sobre as atividades realizadas e apoiar o acompanhamento pela equipe responsável.

1.4 Objetivos específicos

- Identificar as informações necessárias para o planejamento e registro das ações de promoção da saúde mental no ambiente escolar.
- Desenvolver funcionalidades para cadastrar, consultar e acompanhar as ações de promoção da saúde mental realizadas no ambiente escolar.
- Organizar informações sobre as ações e a participação dos estudantes, gerando um resumo que auxilie a equipe no acompanhamento das atividades.

1.5 Justificativa

A motivação para o desenvolvimento deste projeto nasce da nossa percepção, enquanto estudantes e cidadãos, de que o ambiente escolar vai muito além da sala de aula: ele é um espaço fundamental para a formação humana e para o cuidado com o bem-estar dos alunos. No cenário atual, questões como ansiedade, estresse e outras fragilidades emocionais têm impactado cada vez mais a vida de crianças e jovens, tornando a pauta da saúde mental uma prioridade urgente nas escolas. Embora o Programa Saúde na Escola (PSE) contemple ações voltadas a esse tema, percebemos que, na prática, o planejamento e o acompanhamento das atividades ainda sofrem com a falta de organização, dependendo de anotações soltas ou planilhas descentralizadas (SOUSA; ESPERIDIÃO; MEDINA, 2017). Essa rotina acaba transformando ações essenciais em eventos isolados e impede que as equipes entendam a real cobertura das iniciativas.

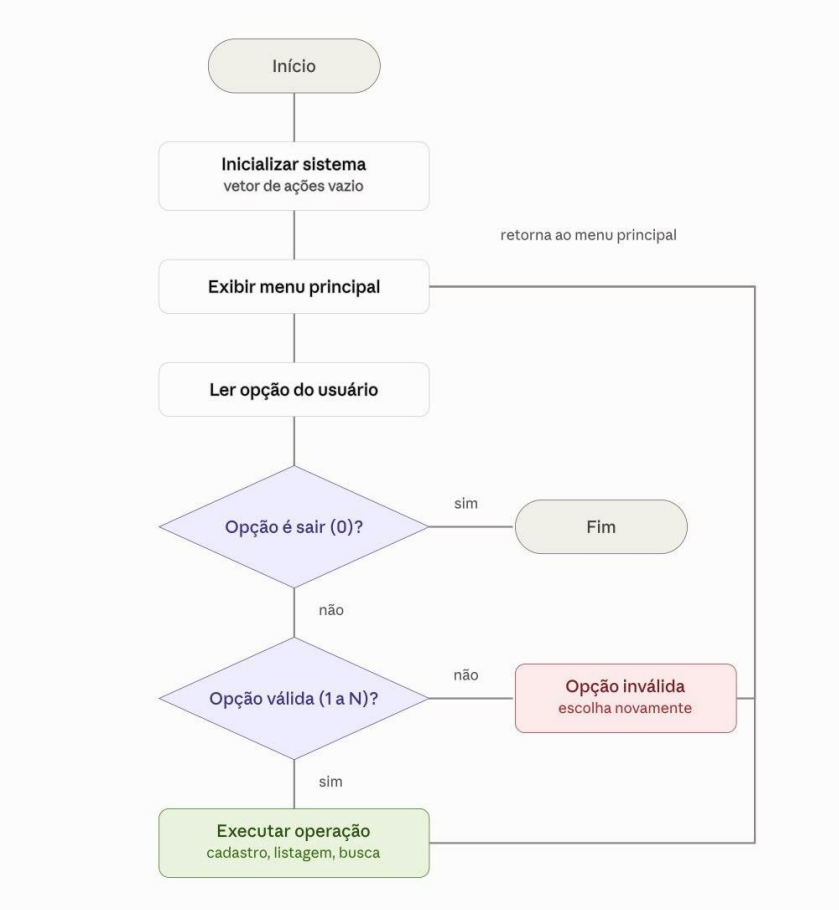
Nossa proposta busca enfrentar essa fragilidade social ao oferecer uma ferramenta que simplifique o dia a dia das equipes de saúde e educação, permitindo registrar, organizar e acompanhar as ações de saúde mental de forma contínua e visual. Com informações claras sobre a participação e o status de cada atividade, fica mais fácil identificar quais turmas ou escolas precisam de mais atenção e planejar intervenções mais eficientes. Ao mesmo tempo, preocupamo-nos em construir uma solução ética: por lidar exclusivamente com dados coletivos e informações fictícias, o sistema preserva a privacidade e a individualidade dos estudantes, garantindo o respeito exigido pelas diretrizes de proteção de dados e de saúde pública (BRASIL, 2025a).

Do ponto de vista acadêmico, o trabalho se justifica por ser a oportunidade de aplicarmos na prática aquilo que aprendemos nas disciplinas do curso, aproximando a teoria da tecnologia de um problema real da nossa comunidade. Construir uma aplicação funcional em Linguagem C nos desafia a colocar em prática conceitos de lógica de programação, o uso de structs, vetores, laços e modularização com funções, além de exigir validações cuidadosas para garantir a estabilidade do programa. Paralelamente, vivenciar as etapas da Engenharia de Software — desde o levantamento dos requisitos e desenho dos fluxogramas até a organização em sprints — desenvolve nossa capacidade de trabalhar em equipe e planejar projetos computacionais. Assim, o desenvolvimento desta aplicação demonstra como a ciência da computação pode ser uma aliada poderosa no fortalecimento de políticas públicas e da cooperação entre Saúde e Educação (CHIARI et al., 2018; VIEIRA; BELISÁRIO, 2018).

2 ALGORITMOS E LÓGICA DE PROGRAMAÇÃO

2.1 Fluxograma Geral

Figura 1 – Fluxograma geral do sistema (menu principal)



Fonte: Elaborado pelas autoras (2026).

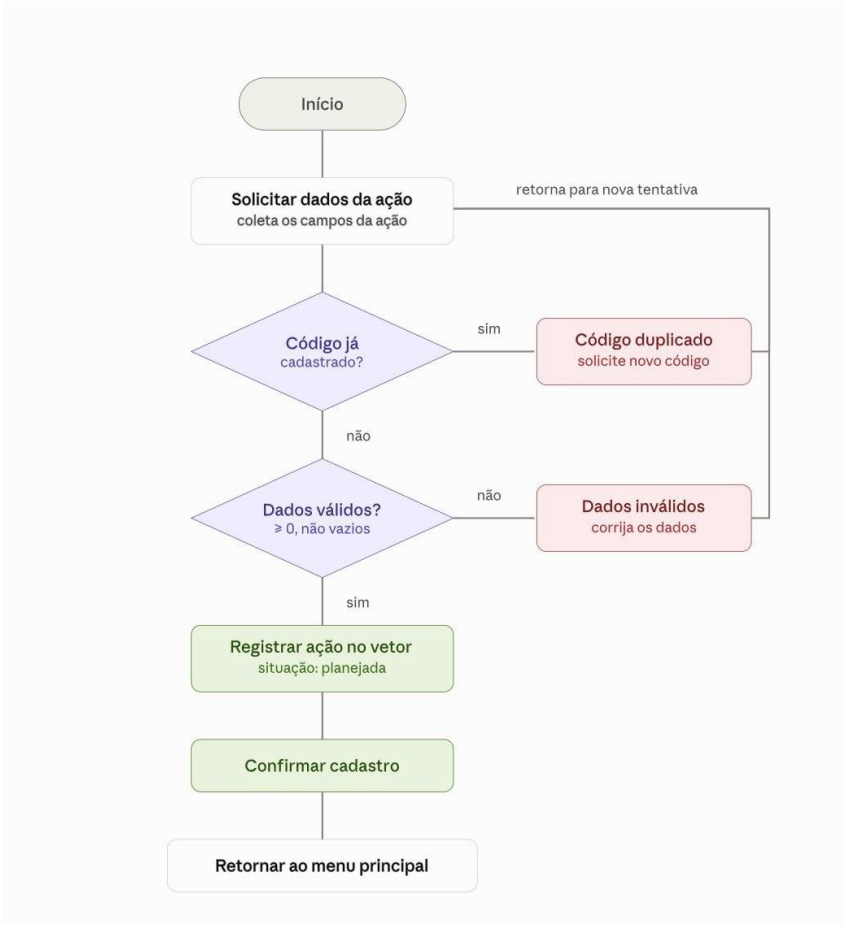
2.2 Pseudocódigo Geral

```
programa
{
  funcao inicio()
  {
    inteiro opcao
    opcao = -1
    enquanto (opcao != 0)
    {
      escreva("\n===== MENTE EM AÇÃO =====\n")
      escreva("1 - Cadastrar ação\n")
      escreva("2 - Listar ações\n")
      escreva("3 - Pesquisar ação\n")
      escreva("4 - Atualizar situação\n")
      escreva("5 - Registrar participantes\n")
      escreva("6 - Gerar resumo\n")
      escreva("0 - Sair\n")
      escreva("\nDigite uma opção: ")
      leia(opcao)
      escolha (opcao)
      {
        caso 1:
          escreva("\nExecutando cadastro...\n")
```

```
            pare
        caso 2:
            escreva("\nExecutando listagem...\n")
            pare
        caso 3:
            escreva("\nExecutando pesquisa...\n")
            pare
        caso 4:
            escreva("\nExecutando atualização de situação...\n")
            pare
        caso 5:
            escreva("\nRegistrando participantes...\n")
            pare
        caso 6:
            escreva("\nGerando resumo...\n")
            pare
        caso 0:
            escreva("\nSistema encerrado!\n")
            pare
        caso contrario:
            escreva("\nOpção inválida! Tente novamente.\n")
    }
  }
 }
}
```

2.3 Fluxograma Específico

Figura 2 – Fluxograma específico da operação de cadastro de ação



Fonte: Elaborado pelas autoras (2026).

2.4 Pseudocódigo Específico

```
programa
{
  funcao inicio()
  {
    inteiro codigo, quantidade, i, total, quantidades[100]
    logico codigoDuplicado
    cadeia escola, tema, data, publico, responsavel, situacao
    inteiro codigos[100]
    cadeia escolas[100], temas[100], datas[100], publicos[100],
           responsaveis[100], situacoes[100]
    total = 0
    escreva("===== CADASTRO DE AÇÃO =====\n\n")
    escreva("Digite o código da ação: ")
    leia(codigo)
    codigoDuplicado = falso
    para (i = 0; i < total; i++)
    {
      se (codigos[i] == codigo)
      {
        codigoDuplicado = verdadeiro
      }
    }
    se (codigoDuplicado)
    {
      escreva("\nCódigo duplicado!\n")
      escreva("Cadastro não realizado.\n")
    }
    senao
    {
      escreva("Digite o nome da escola: ")
      leia(escola)
      escreva("Digite o tema da ação: ")
      leia(tema)
      escreva("Digite a data da ação: ")
      leia(data)
      escreva("Digite o público-alvo: ")
      leia(publico)
      escreva("Digite o responsável pela ação: ")
      leia(responsavel)
      escreva("Digite a quantidade prevista de participantes: ")
      leia(quantidade)
      se (escola == "" ou tema == "" ou data == "" ou publico == "" ou
responsavel == "" ou quantidade <= 0)
      {
        escreva("\nDados inválidos!\n")
        escreva("Verifique os campos informados.\n")
      }
      senao
      {
        codigos[total] = codigo
        escolas[total] = escola
        temas[total] = tema
        datas[total] = data
        publicos[total] = publico
        responsaveis[total] = responsavel
        quantidades[total] = quantidade
        situacao = "PLANEJADA"
        situacoes[total] = situacao
        total = total + 1
        escreva("\nCadastro realizado com sucesso!\n")
        escreva("Situação inicial: PLANEJADA\n")
      }
    }
```

```
  }
}
```

3 ENGENHARIA DE SOFTWARE

3.1 Identificação dos usuários envolvidos

- **Equipe Intersetorial (Saúde e Educação):** Profissionais da saúde (psicólogos, enfermeiros e assistentes sociais) e das escolas (secretaria, diretores e professores) responsáveis pelo planejamento, cadastro e utilização das oficinas e palestras de saúde mental.
- **Coordenadores do PSE:** Gestores públicos que utilizam o sistema para acompanhar os relatórios, verificar o alcance das ações nas escolas e tomar decisões com base nos dados.

3.2 Delimitação do Escopo do Sistema

O Mente em Ação será um programa desenvolvido em linguagem C e executado pelo terminal. A proposta é auxiliar no planejamento e no acompanhamento de ações voltadas à promoção da saúde mental no ambiente escolar.

- O sistema irá permitir cadastrar novas ações, visualizar as atividades já registradas, realizar buscas, alterar a situação das ações e consultar um resumo com os resultados obtidos.
- Para cada ação cadastrada, serão armazenadas algumas informações importantes, como código, escola, tema, data prevista, público-alvo, responsável, número esperado de participantes e situação da atividade.
- As ações poderão apresentar três situações diferentes: planejada, realizada ou cancelada. Nos casos em que a atividade for realizada, o sistema também permitirá informar quantas pessoas participaram efetivamente.

O projeto terá algumas limitações. Não serão utilizados banco de dados, interface gráfica ou aplicação web. Além disso, o programa não terá funções relacionadas a diagnóstico, tratamento ou registro de informações clínicas de pessoas. Os dados ficarão armazenados apenas durante a execução do programa.

3.3 Requisitos Funcionais

RF01 – Cadastrar uma nova ação de promoção da saúde mental, registrando código, escola, tema, data prevista, público-alvo, responsável e número esperado de participantes.

RF02 – Listar todas as ações cadastradas, exibindo seus principais dados de forma organizada.

RF03 – Buscar uma ação específica cadastrada a partir do código informado pelo usuário.

RF04 – Alterar a situação de uma ação cadastrada entre planejada, realizada ou cancelada.

RF05 – Registrar o número de participantes efetivos quando uma ação for marcada como realizada.

RF06 – Gerar um resumo com a quantidade de ações por situação e o total de participantes nas ações realizadas.

RF07 – Permitir que o usuário encerre a execução do programa de forma segura.

3.4 Requisitos Não Funcionais

RNF01 – O sistema deve ser desenvolvido em linguagem C e executado em ambiente de terminal (linha de comando).

RNF02 – O sistema deve responder às operações do usuário de forma imediata, sem necessidade de processamento externo.

RNF03 – O sistema não deve utilizar banco de dados, mantendo os dados armazenados apenas durante o tempo de execução do programa.

RNF04 – O código-fonte deve ser modularizado em funções, facilitando a leitura, a manutenção e a reutilização.

RNF05 – O sistema deve validar as entradas fornecidas pelo usuário, impedindo valores inválidos, como número de participantes negativo ou situação inexistente.

RNF06 – O sistema não deve armazenar dados pessoais ou clínicos identificáveis dos estudantes, preservando a privacidade e utilizando apenas informações coletivas e fictícias.

3.5 Processo de desenvolvimento

Para o desenvolvimento do sistema Mente em Ação, foi adotado o processo de desenvolvimento incremental, no qual o software é construído e entregue em pequenas partes funcionais ao longo do tempo, em vez de ser desenvolvido e entregue de uma só vez apenas ao final do projeto. A cada semana, é implementado um novo incremento contendo uma ou mais funcionalidades do sistema, permitindo testar, corrigir e validar cada parte antes de avançar para a etapa seguinte (SOMMERVILLE, 2018).

Essa abordagem foi escolhida por se adequar à realidade de uma equipe pequena e ao prazo reduzido do projeto, possibilitando entregas semanais que facilitam o acompanhamento do progresso, a identificação antecipada de erros e o

ajuste do planejamento sempre que necessário. Dessa forma, ao final de cada sprint, a equipe passa a dispor de uma versão do programa mais completa e funcional, aproximando o desenvolvimento do projeto das práticas ágeis de engenharia de software.

3.6 Planejamento das Sprints

O desenvolvimento do sistema foi planejado em quatro sprints semanais, cada uma contemplando etapas de levantamento, modelagem, implementação, testes, documentação e, na última semana, a preparação da apresentação final, conforme detalhado a seguir:

- **Sprint 1 (Semana 1) – Levantamento e modelagem**
  - Levantamento dos requisitos funcionais e não funcionais junto à equipe.
  - Definição da estrutura de dados (struct) da ação de saúde mental.
  - Elaboração dos fluxogramas e do pseudocódigo das principais funcionalidades.
  - Documentação do escopo e dos requisitos do sistema.
- **Sprint 2 (Semana 2) – Implementação do cadastro**
  - Implementação da função de cadastro de novas ações.
  - Implementação da função de listagem das ações cadastradas.
  - Testes iniciais das funcionalidades implementadas.
- **Sprint 3 (Semana 3) – Busca, atualização e validações**
  - Implementação da função de busca de ações pelo código.
  - Implementação da função de alteração da situação das ações (planejada, realizada ou cancelada).
  - Implementação das validações de entrada de dados.
  - Testes das novas funcionalidades e correção de erros.
- **Sprint 4 (Semana 4) – Resumo, testes finais e apresentação**
  - Implementação da função de geração do resumo das ações.
  - Testes finais e revisão geral do sistema.
  - Finalização da documentação do projeto.
  - Preparação do material e ensaio da apresentação final.

A análise dos riscos do projeto e a atualização dos requisitos estão na seção 5, que apresenta a 2ª etapa da AEP.

4 IMPLEMENTAÇÃO EM LINGUAGEM C

As seções 4 a 8 apresentam a 2ª etapa da AEP: a implementação do sistema, os artefatos de Engenharia de Software revisados, os testes, o manual do usuário, as considerações finais e a apresentação em vídeo.

Repositório no GitHub: https://github.com/nathaliabea/AEP_MENTE_EM_ACAO

O sistema foi implementado em um único arquivo-fonte (mente_em_acao.c), com armazenamento em memória por meio de um vetor de structs com capacidade para 100 ações. O código-fonte completo está disponível no repositório do projeto no GitHub, indicado acima.

5 ENGENHARIA DE SOFTWARE – 2ª ETAPA

5.1 Requisitos funcionais (atualizados)

- **RF01 –** Cadastrar ação com código, escola, tema, data prevista, público-alvo, responsável, quantidade prevista e situação inicial “planejada”.
- **RF02 –** Listar todas as ações cadastradas de forma organizada.
- **RF03 –** Pesquisar ações por código, escola ou tema. (Atualizado: na 1ª entrega a busca era somente por código; o desafio da AEP exige também escola e tema.)
- **RF04 –** Atualizar a situação de uma ação entre planejada, realizada ou cancelada.
- **RF05 –** Registrar a quantidade efetiva de participantes quando a ação for realizada.
- **RF06 –** Gerar resumo com quantidade de ações por situação, total de participantes e percentual de participação em relação ao previsto. (Atualizado: inclusão do percentual.)
- **RF07 –** Encerrar o programa de forma segura pelo menu.

5.2 Requisitos não funcionais

- **RNF01 –** Desenvolvimento em linguagem C, executado em terminal.
- **RNF02 –** Resposta imediata às operações, sem processamento externo.
- **RNF03 –** Sem banco de dados; dados mantidos apenas em memória durante a execução.
- **RNF04 –** Código modularizado em funções com responsabilidades claras.
- **RNF05 –** Validação das entradas (código repetido, campos vazios, quantidade negativa, opção inexistente no menu, registro inexistente e limite de armazenamento).

- **RNF06 –** Nenhum dado pessoal ou clínico identificável; somente informações coletivas e fictícias (BRASIL, 2018).
- **RNF07 –** Compilação e execução no Dev-C++ (GCC/MinGW), com bibliotecas padrão da linguagem C. (Novo na 2ª etapa.)

5.3 Análise de riscos do projeto

Os riscos foram identificados e avaliados quanto à probabilidade e ao impacto, com estratégias de mitigação, conforme as diretrizes de gestão de riscos da ABNT NBR ISO 31000 (ABNT, 2018).

Quadro 1 – Análise de riscos do projeto

ID	Risco	Prob.	Impacto	Estratégia de mitigação
R01	Atraso no cronograma das sprints por falta de tempo da equipe	Média	Alto	Entregas semanais curtas, divisão clara de tarefas e prioridade às funcionalidades do escopo mínimo
R02	Erros de lógica ou falhas do programa (travamentos, loops infinitos)	Média	Alto	Testes a cada sprint, revisão do código em dupla e registro dos testes
R03	Entradas inválidas do usuário (letras no lugar de números, campos vazios)	Alta	Médio	Validação de todas as entradas e limpeza do buffer de leitura
R04	Perda de dados ao encerrar o programa (armazenamento só em memória)	Alta	Médio	Declarar a limitação no manual; propor gravação em arquivo como melhoria futura
R05	Limite do vetor de ações atingido	Média	Médio	Constante de limite, aviso ao usuário e tratamento do caso no cadastro
R06	Perda ou conflito de versões do código	Baixa	Alto	Repositório no GitHub com commits frequentes e README atualizado
R07	Uso indevido com dados pessoais ou clínicos de estudantes	Baixa	Alto	Campos coletivos apenas, dados fictícios e aviso no manual (BRASIL, 2018)
R08	Desencontro entre requisitos e implementação	Média	Médio	Revisão dos requisitos a cada sprint e registro das mudanças justificadas
R09	Falha na gravação ou no acesso ao vídeo de apresentação	Baixa	Médio	Ensaio prévio, limite de 7 minutos e conferência da liberação do link
R10	Código de ação duplicado ou busca por registro inexistente	Média	Médio	Verificar duplicidade antes de gravar e exibir mensagem de registro não encontrado
R11	Uso das informações como diagnóstico ou triagem	Baixa	Alto	Delimitar o escopo (sem informações clínicas) e registrar essa limitação na documentação e no vídeo
R12	Pouca experiência da equipe com strings, struct e vetores em C	Média	Médio	Começar pelas funções mais simples, revisar o código em dupla e consultar o professor
R13	Programa não compilar no ambiente do professor	Baixa	Alto	Usar C padrão, testar no Dev-C++ e com GCC e informar a forma de compilar no README

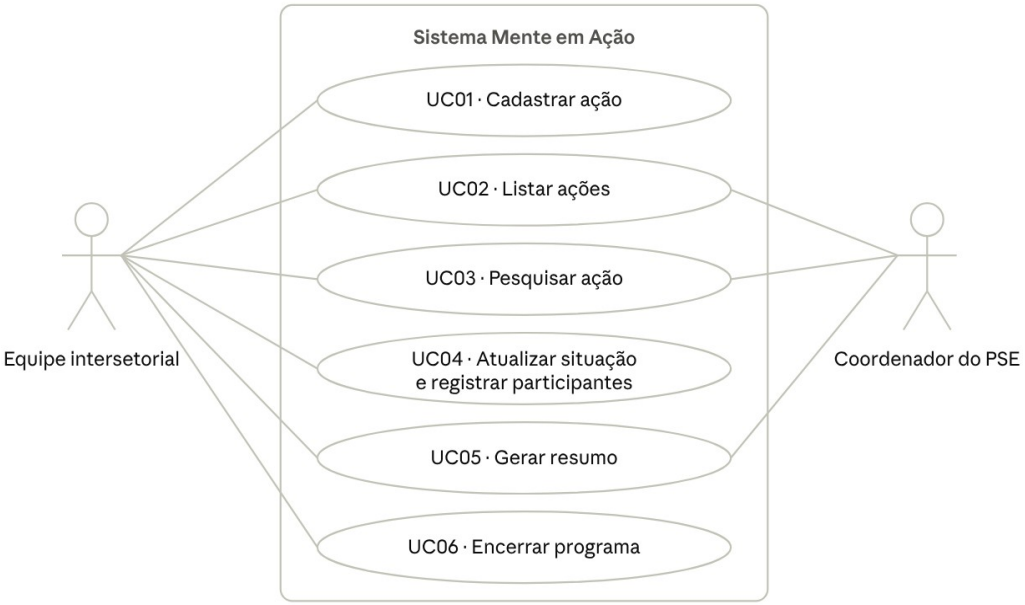
Fonte: Elaborado pelas autoras (2026).

5.4 Casos de uso

Os casos de uso seguem a notação da UML (GUEDES, 2018).

5.4.1 Diagrama de casos de uso

Figura 3 – Diagrama de casos de uso do sistema Mente em Ação



Fonte: Elaborado pelas autoras (2026).

A equipe intersetorial participa dos seis casos de uso; o coordenador do PSE consulta, pesquisa e gera o resumo.

5.4.2 Descrição dos casos de uso principais

UC01 – Cadastrar ação

Campo	Descrição
Objetivo	Registrar uma nova ação de saúde mental.
Ator	Usuário (equipe intersetorial / coordenador do PSE)
Pré-condições	Programa em execução; limite de armazenamento não atingido.
Fluxo principal	1. O usuário escolhe a opção 1. 2. Informa código, escola, tema, data, público-alvo, responsável e quantidade prevista. 3. O sistema valida os dados. 4. O sistema registra a ação como “planejada” e confirma.
Fluxos alternativos	A1: código já existente → o sistema informa duplicidade e cancela o cadastro. A2: campo vazio ou quantidade menor ou igual a zero → o sistema informa dados inválidos. A3: limite atingido → o sistema avisa que não há espaço.
Pós-condições	Ação disponível para listagem, pesquisa e atualização.

UC02 – Listar ações

Campo	Descrição
Objetivo	Exibir todas as ações cadastradas.
Ator	Usuário (equipe intersetorial / coordenador do PSE)
Pré-condições	Programa em execução.
Fluxo principal	1. O usuário escolhe a opção 2. 2. O sistema exibe os dados de cada ação em formato organizado.
Fluxos alternativos	A1: nenhuma ação cadastrada → o sistema informa que a lista está vazia.
Pós-condições	Nenhum dado é alterado.

UC03 – Pesquisar ação

Campo	Descrição
Objetivo	Localizar ações por código, escola ou tema.
Ator	Usuário (equipe intersetorial / coordenador do PSE)
Pré-condições	Ao menos uma ação cadastrada.
Fluxo principal	1. O usuário escolhe a opção 3. 2. Seleciona o critério (código, escola ou tema) e informa o valor. 3. O sistema exibe os registros encontrados.
Fluxos alternativos	A1: nenhum registro encontrado → o sistema informa registro inexistente. A2: critério inválido → o sistema informa o erro e retorna ao menu.
Pós-condições	Nenhum dado é alterado.

UC04 – Atualizar situação e registrar participantes

Campo	Descrição
Objetivo	Alterar a situação da ação e, se realizada, registrar os participantes efetivos.
Ator	Usuário (equipe intersetorial / coordenador do PSE)
Pré-condições	Ação existente.
Fluxo principal	1. O usuário escolhe a opção 4 (atualizar situação) ou a opção 5 (registrar participantes). 2. Informa o código da ação. 3. Na opção 4, escolhe a nova situação (planejada, realizada ou cancelada); se “realizada”, informa a quantidade efetiva. 4. O sistema atualiza e confirma.
Fluxos alternativos	A1: código inexistente → o sistema informa e retorna ao menu. A2: situação inexistente → o sistema informa o erro e mantém a situação. A3: quantidade negativa → o sistema rejeita a entrada. A4: opção 5 em ação que não está realizada → o sistema orienta a alterar antes a situação.
Pós-condições	Situação e participantes atualizados.

UC05 – Gerar resumo

Campo	Descrição
Objetivo	Consolidar os resultados das ações.
Ator	Usuário (equipe intersetorial / coordenador do PSE)
Pré-condições	Ao menos uma ação cadastrada. Ator: equipe intersetorial ou coordenador do PSE.
Fluxo principal	1. O usuário escolhe a opção 6. 2. O sistema calcula e exibe a quantidade de ações planejadas, realizadas e canceladas, o total de participantes e o percentual de participação em relação ao previsto.
Fluxos alternativos	A1: nenhuma ação cadastrada → o sistema informa que não há dados. A2: sem ações realizadas → o sistema informa que o percentual não se aplica.
Pós-condições	Nenhum dado é alterado.

UC06 – Encerrar programa

Campo	Descrição
Objetivo	Finalizar a execução com segurança.
Ator	Usuário (equipe intersetorial / coordenador do PSE)
Pré-condições	Programa em execução.
Fluxo principal	1. O usuário escolhe a opção 0. 2. O sistema exibe a mensagem de encerramento e finaliza.
Fluxos alternativos	A1: opção inexistente em qualquer momento → o sistema informa “Opção inválida” e reexibe o menu.
Pós-condições	Dados em memória são descartados.

5.5 Registro de testes

Os testes abaixo cobrem as principais funcionalidades e validações. Foram executados com o programa compilado com GCC (mesmo compilador usado pelo Dev-C++).

Quadro 2 – Registro de testes realizados

ID	Funcionalidade	Entrada	Resultado esperado	Resultado obtido	Situação
T01	Cadastro válido	Dados completos, código novo	Cadastro com situação “planejada”	“Cadastro realizado com sucesso! Situacao inicial: PLANEJADA”	Aprovado
T02	Código duplicado	Código já cadastrado	Mensagem de duplicidade; sem cadastro	“Codigo duplicado!” e cadastro não realizado	Aprovado
T03	Campo vazio	Escola em branco	Mensagem de dados inválidos	“Dados invalidos! Todos os campos sao obrigatorios.”	Aprovado

ID	Funcionalidade	Entrada	Resultado esperado	Resultado obtido	Situação
T04	Quantidade inválida	Quantidade prevista -5	Mensagem de dados inválidos	“Quantidade invalida! Deve ser maior que zero.”	Aprovado
T05	Limite de armazenamento	Cadastrar o 101º registro	Aviso de limite atingido	“Limite de 100 acoes atingido. Cadastro nao realizado.”	Aprovado
T06	Listagem vazia	Opção 2 sem ações	Aviso de lista vazia	“Nenhuma acao cadastrada.”	Aprovado
T07	Pesquisa por código	Código existente	Exibe a ação	Ação exibida com todos os dados	Aprovado
T08	Pesquisa por escola/tema	Escola “Beta” e tema “Roda”	Exibe as ações correspondentes	Ações correspondentes exibidas (busca por trecho do nome)	Aprovado
T09	Pesquisa inexistente	Código 77 não cadastrado	Mensagem de registro inexistente	“Registro inexistente: nenhuma acao com o codigo 77.”	Aprovado
T10	Atualizar para realizada	Código válido e 30 participantes	Situação e participantes atualizados	Situação REALIZADA com 30 participantes	Aprovado
T11	Participantes negativos	Quantidade -3	Entrada rejeitada	“Quantidade invalida!” e “Situacao nao alterada.”	Aprovado
T12	Resumo	Opção 6 com 1 planejada, 1 realizada (30 de 40) e 1 cancelada	Totais e percentual corretos	1/1/1; 30 participantes; 75,0%	Aprovado
T13	Opção inválida no menu	Digitar 9	“Opção inválida” e menu reexibido	“Opcao invalida! Escolha uma opcao do menu.”	Aprovado
T14	Encerrar	Opção 0	Mensagem de encerramento	“Sistema encerrado!”	Aprovado
T15	Texto no menu	Digitar “abc”	Mensagem de erro, sem travar	“Opcao invalida! Escolha uma opcao do menu.”	Aprovado
T16	Registrar participantes	Opção 5 em ação realizada, valor 35	Quantidade atualizada	“Participantes registrados com sucesso!”; resumo com 87,5%	Aprovado
T17	Registrar participantes	Opção 5 em ação planejada	Entrada rejeitada com orientação	“A acao esta PLANEJADA. Altere a situacao para REALIZADA (opcao 4) antes.”	Aprovado

Fonte: Elaborado pelas autoras (2026).

6 MANUAL DO USUÁRIO

6.1 Objetivo

O Mente em Ação auxilia equipes de saúde e educação a cadastrar, consultar e acompanhar ações coletivas de promoção da saúde mental nas escolas, sem armazenar dados pessoais ou clínicos. Ele se insere no Programa Saúde na Escola, instituído pelo Decreto nº 6.286/2007 (BRASIL, 2007), cuja base é a articulação entre a escola e a Atenção Primária (BRASIL, 2025a) e que inclui a promoção da saúde mental entre suas ações (BRASIL, 2025b).

Atenção: o sistema trabalha somente com dados fictícios e coletivos. Não cadastre nome, diagnóstico ou qualquer informação individual de estudantes. Os dados ficam apenas na memória e são perdidos quando o programa é encerrado.

6.2 Compilação e execução

No Dev-C++:

- Abra o Dev-C++ e vá em Arquivo > Abrir Arquivo, selecionando mente_em_acao.c (ou crie um novo arquivo-fonte e cole o código).
- Se for um arquivo novo, salve com a extensão .c.
- Pressione F11 (Compilar e Executar). O terminal será aberto com o menu do programa.

Pelo terminal (GCC):

```
gcc mente_em_acao.c -o mente_em_acao
./mente_em_acao          (Linux/macOS)
mente_em_acao.exe        (Windows)
```

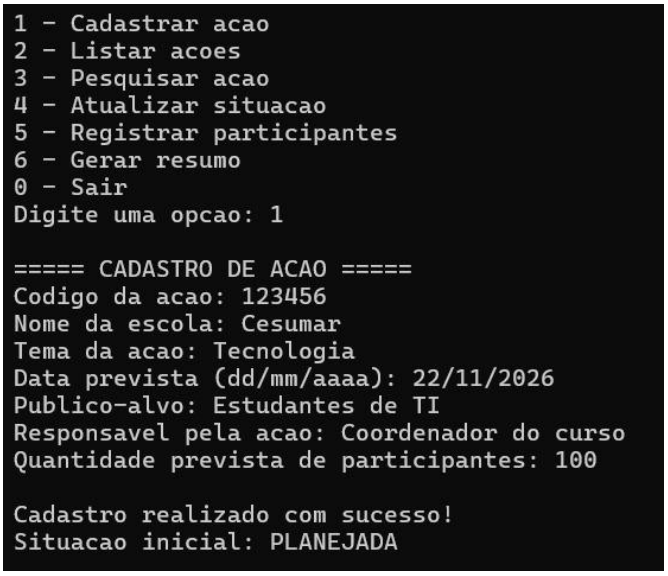
6.3 Opções do menu

Opção	Função	O que informar
1	Cadastrar ação	Código (número positivo e único), escola, tema, data (dd/mm/aaaa), público-alvo, responsável e quantidade prevista de participantes
2	Listar ações	Nada; o sistema mostra todas as ações cadastradas
3	Pesquisar ação por código, escola ou tema	Critério (1 código, 2 escola, 3 tema) e o termo de busca; para escola e tema basta um trecho do nome, respeitando maiúsculas e minúsculas
4	Atualizar situação (planejada, realizada ou cancelada)	Código da ação e nova situação (1 planejada, 2 realizada, 3 cancelada); se realizada, a quantidade efetiva de participantes
5	Registrar participantes efetivos	Código de uma ação já realizada e a quantidade efetiva de participantes
6	Gerar resumo geral	Nada; o sistema mostra o resumo geral
0	Sair	Nada; encerra o programa

6.4 Exemplos de uso e capturas de tela

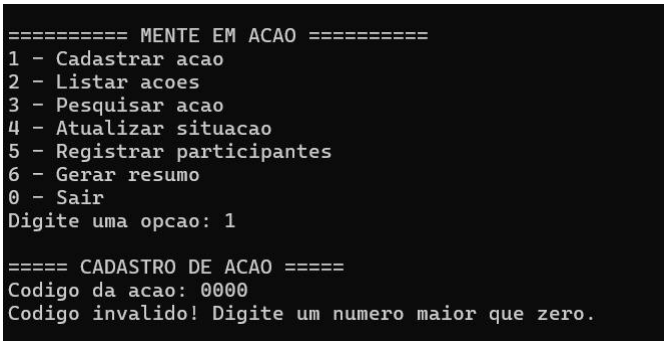
As capturas de tela a seguir mostram o programa em execução no Dev-C++, com as principais operações e validações do sistema.

Figura 4 – Cadastro de uma ação (opção 1)



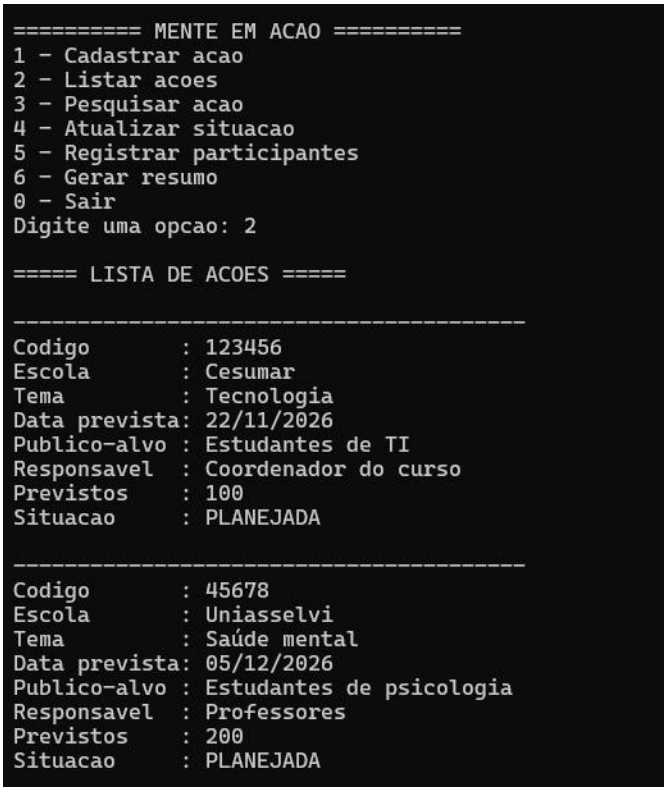
Fonte: Elaborado pelas autoras (2026).

Figura 5 – Validação de código inválido no cadastro



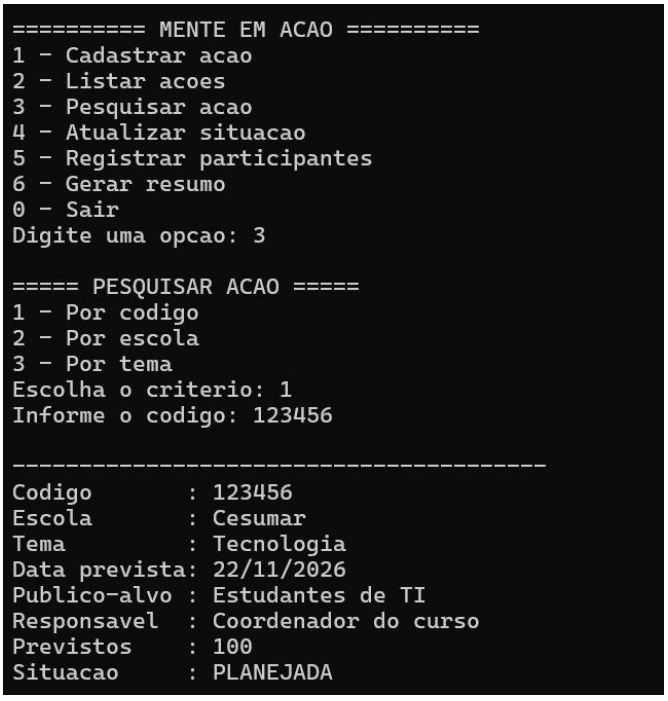
Fonte: Elaborado pelas autoras (2026).

Figura 6 – Listagem das ações cadastradas (opção 2)



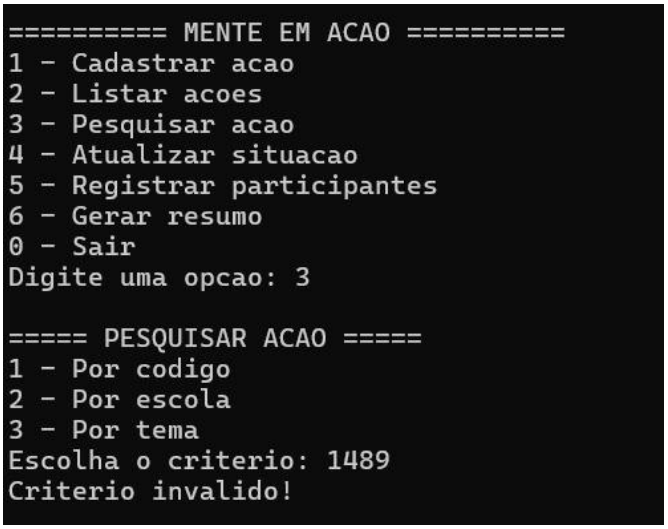
Fonte: Elaborado pelas autoras (2026).

Figura 7 – Pesquisa de ação por código (opção 3)



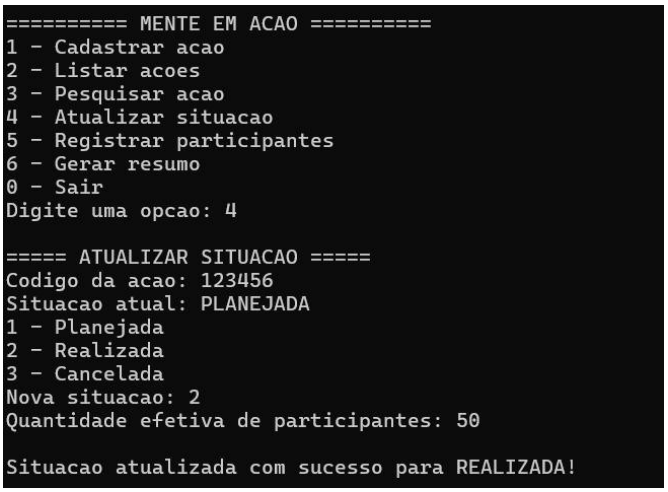
Fonte: Elaborado pelas autoras (2026).

Figura 8 – Validação de critério de pesquisa inválido



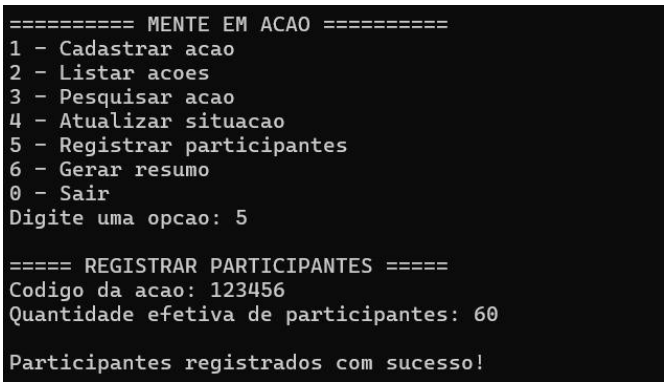
Fonte: Elaborado pelas autoras (2026).

Figura 9 – Atualização da situação para realizada (opção 4)



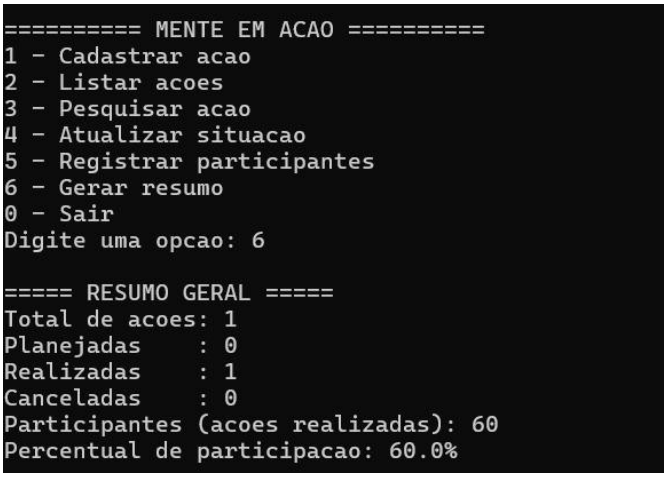
Fonte: Elaborado pelas autoras (2026).

Figura 10 – Registro de participantes efetivos (opção 5)



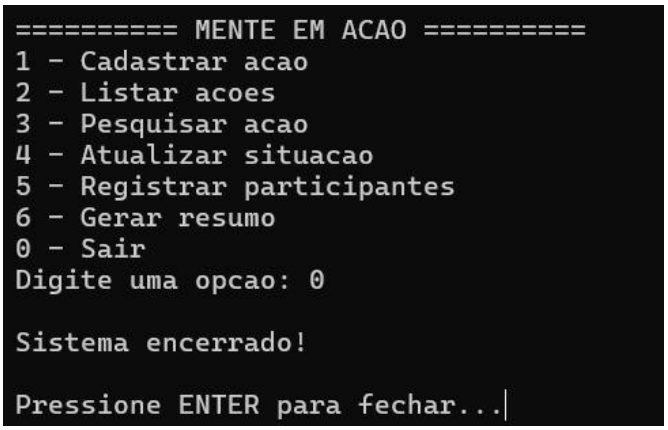
Fonte: Elaborado pelas autoras (2026).

Figura 11 – Resumo geral com percentual de participação (opção 6)



Fonte: Elaborado pelas autoras (2026).

Figura 12 – Encerramento do programa (opção 0)



Fonte: Elaborado pelas autoras (2026).

6.5 Mensagens de erro mais comuns

Mensagem	Causa	O que fazer
Opcao invalida!	Opção fora do menu ou texto no lugar de número	Digite um número do menu
Codigo duplicado!	Já existe ação com o código informado	Use outro código ou pesquise a ação existente
Dados invalidos!	Algum campo obrigatório ficou vazio	Preencha todos os campos
Quantidade invalida!	Valor não numérico, zero (previstos) ou negativo	Informe um número inteiro válido
Registro inexistente	Não há ação com o código informado	Confira o código na listagem
Limite de 100 acoes atingido	O vetor de ações está cheio	Reinicie o programa ou aumente MAX_ACOES no código

7 CONSIDERAÇÕES FINAIS

Resultados: o sistema atende ao escopo mínimo do desafio: cadastra, lista, pesquisa por código, escola ou tema, atualiza a situação, registra participantes e gera resumo com percentual de participação, com validação das entradas. Isso dialoga com a necessidade, apontada por Sousa, Esperidião e Medina (2017), de organizar e acompanhar as ações do PSE, que muitas vezes ficam pontuais. Dessa forma, a aplicação responde à questão norteadora do trabalho ao apoiar o planejamento, o registro e o acompanhamento das ações de saúde mental sem armazenar dados pessoais ou clínicos.

Limitações: os dados ficam apenas em memória e se perdem ao encerrar; há limite fixo de 100 ações; a interface é de terminal; não há controle de usuários nem opção de editar ou excluir um cadastro. O sistema trabalha somente com dados fictícios e coletivos, o que atende ao cuidado com dados pessoais previsto na LGPD (BRASIL, 2018).

Aprendizados: aplicação prática de structs, vetores, funções e validações; uso de casos de uso na modelagem, conforme a UML (GUEDES, 2018); análise de riscos baseada na ABNT NBR ISO 31000 (ABNT, 2018); desenvolvimento incremental (SOMMERVILLE, 2018); e trabalho em equipe com GitHub.

Melhorias futuras: gravação e leitura em arquivo, relatórios por escola e por período, edição e exclusão de ações, e interface mais amigável.

8 APRESENTAÇÃO EM VÍDEO

Link do vídeo (até 7 minutos, acesso liberado para avaliação): https://youtu.be/F5rGUPHYIhk?is=g1NTUt6AFGmKWwON

Link do GitHub: https://github.com/nathaliabea/AEP_MENTE_EM_ACAO

REFERÊNCIAS

ABNT. NBR ISO 31000:2018 – Gestão de riscos – Diretrizes. Rio de Janeiro: ABNT, 2018.

BRASIL. Decreto nº 6.286, de 5 de dezembro de 2007. Institui o Programa Saúde na Escola – PSE e dá outras providências. Diário Oficial da União: Brasília, DF, 6 dez. 2007.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: Brasília, DF, 15 ago. 2018.

BRASIL. Ministério da Saúde. Diretrizes para implantação do Projeto Saúde e Prevenção nas Escolas. Brasília, DF, 2006.

BRASIL. Ministério da Saúde. O que é o Programa Saúde na Escola (PSE)? Brasília, DF, 2025a. Disponível no portal do Ministério da Saúde. Acesso em: 24 jul. 2026.

BRASIL. Ministério da Saúde. Quais são as ações do PSE? Brasília, DF, 2025b. Disponível no portal do Ministério da Saúde. Acesso em: 24 jul. 2026.

BRASIL. Portaria Interministerial nº 1.055, de 25 de abril de 2017. Redefine as regras e os critérios para adesão ao Programa Saúde na Escola – PSE. Brasília, DF, 2017.

CHIARI, Antônio Paulo Gomes et al. Rede intersetorial do Programa Saúde na Escola: sujeitos, percepções e práticas. Cadernos de Saúde Pública, Rio de Janeiro, v. 34, n. 5, p. e00060917, 2018.

GUEDES, Gilleanes T. A. UML 2: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SOUSA, Marta Caires de; ESPERIDIÃO, Monique Azevedo; MEDINA, Maria Guadalupe. A intersetorialidade no Programa Saúde na Escola: avaliação do processo político-gerencial e das práticas de trabalho. Ciência & Saúde Coletiva, Rio de Janeiro, v. 22, n. 6, p. 1781-1790, 2017. DOI: 10.1590/1413-81232017226.24262016.

VIEIRA, Lidiane Sales; BELISÁRIO, Soraya Almeida. Intersetorialidade na promoção da saúde escolar: um estudo do Programa Saúde na Escola. Saúde em Debate, Rio de Janeiro, v. 42, n. 4, p. 120-133, 2018.